

Classport: Designing Runtime Dependency Introspection for Java

Abstract

Runtime introspection of dependencies, i.e., the ability to observe which dependencies are currently used during program execution, is fundamental for Software Supply Chain security. Yet, Java has no support for it. We solve this problem with Classport, a system that embeds build-time dependency information into Java class files and retrieves dependency information at runtime. We evaluate it on six real-world projects, demonstrating its effectiveness in identifying dependencies at runtime.

1 Introduction

Software systems keep increasing in size and complexity. To speed up the implementation and satisfy the growing demand for features, developers often reuse existing libraries [11]. They leverage third-party open-source code, which is integrated into the main application as a dependency. This collection of source code, dependencies, and build tools used to aggregate these two into a main application is known as the Software Supply Chain [11].

Using third-party dependencies is essential for productivity and reliability, but it also increases the attack surface [14]. Security vulnerabilities that exist in a dependency are inherited by downstream software products, leading to potential compromises. Examples of notable Software Supply Chain attacks are the Log4Shell attack [29], the Codecov compromise [24], and the xz catastrophe [20]. Assessing and hardening the security of the Software Supply Chain requires increasing its so-called “transparency”, i.e., the visibility on its components [13].

One of the key tools to support transparency is the production of Software Bill of Materials (SBOM) [8]. An SBOM is an exhaustive record of the individual components included in a software system [12]. Maintainers can use this list as input for vulnerability assessment tools to efficiently detect malicious or vulnerable software packages. Most of the approaches to SBOM operate statically, by looking for dependencies in metadata files, such as manifests or lock files, only at build time [5, 22, 33]. This has two fundamental limitations: 1) it does not reflect the actual dependency usage at runtime, i.e., whether a dependency is actually loaded or used in production [2], 2) it does not support the maintainers or developers to make runtime decisions based on its dependencies. Basically, the fundamental limitation of SBOM is that all dependency information is lost at runtime.

Runtime dependency introspection solves these limitations. It is the ability of a software to observe which components are executed during a program’s actual run. Runtime dependency introspection is the necessary building block for advanced security hardening. First, it is crucial for identifying bloated dependencies, i.e., those that are present in the application but never used. Removing them reduces the attack surface, hence improving the security posture. Second, runtime dependency introspection also plays a fundamental role in restricting privileges at runtime, i.e., determining which resource can be accessed by which dependency.

The Java runtime is known for its good support of object-oriented introspection (aka class, field, and method reflection). Yet, it is

completely blind to dependencies; it has no native support for dependency introspection at runtime [31]. Without this information, developers, users, and maintainers cannot know which and how dependencies are used at runtime. Prior work has explored runtime dependency introspection for specific use cases, such as detecting vulnerable methods [18], or identifying bloated code [26], based on heuristics like guessing the origin of dependencies from package names [1], which is weak. To the best of our knowledge, **there are currently no principled blueprints for identifying which dependencies are actually executed at runtime in Java.**

In this paper, we propose Classport, a novel technique for introspecting dependencies at runtime in Java. In essence, Classport brings dependency information to runtime, information that is today only available at build time. Classport operates in two phases by: (1) embedding dependency information into Java binary artifacts, and (2) retrieving this information at runtime through instrumentation.

We evaluate Classport on six real-world open-source Java projects. Specifically, we determine its ability to embed dependency information into the binary artifacts, assessing the time and disk space overhead caused by the added information. Then, we test Classport’s ability to identify the correct set of runtime dependencies according to a meaningful workload. We also consider the impact on the running application, incl. the runtime overhead.

Our results demonstrate the applicability of Classport to real-world applications. Classport successfully embeds dependency information, accurately identifies runtime dependencies, preserves the functional behavior of the application, and introduces a negligible overhead.

In summary, the main contributions of this paper are:

- Classport, a novel approach to inspect Java dependencies at runtime, based on Java annotations. To the best of our knowledge, it is the first ever generic solution to the key problem of dependency introspection at runtime.
- A publicly available prototype to embed information into binary artifacts and a runtime agent to retrieve dependency information during execution.¹
- An evaluation of our novel technique on a set of six real-world applications, demonstrating the feasibility and applicability of our approach.²

2 Background

This section provides background on how dependency management works in the Java ecosystem, with a focus on Maven and its limitations regarding runtime visibility of dependencies.

Dependency management by package managers and build systems varies across ecosystems, as they handle resolution, versioning, and transitive dependencies differently. In the Java ecosystem, Maven is one of the official build systems [10]. Within Maven, developers declare dependencies in a file called `pom.xml`. Each of these dependencies is downloaded as a binary artifact called a JAR [27],

¹<https://anonymous.4open.science/r/classport-FD6E/>

²<https://anonymous.4open.science/r/classport-experiments-C7DD/>

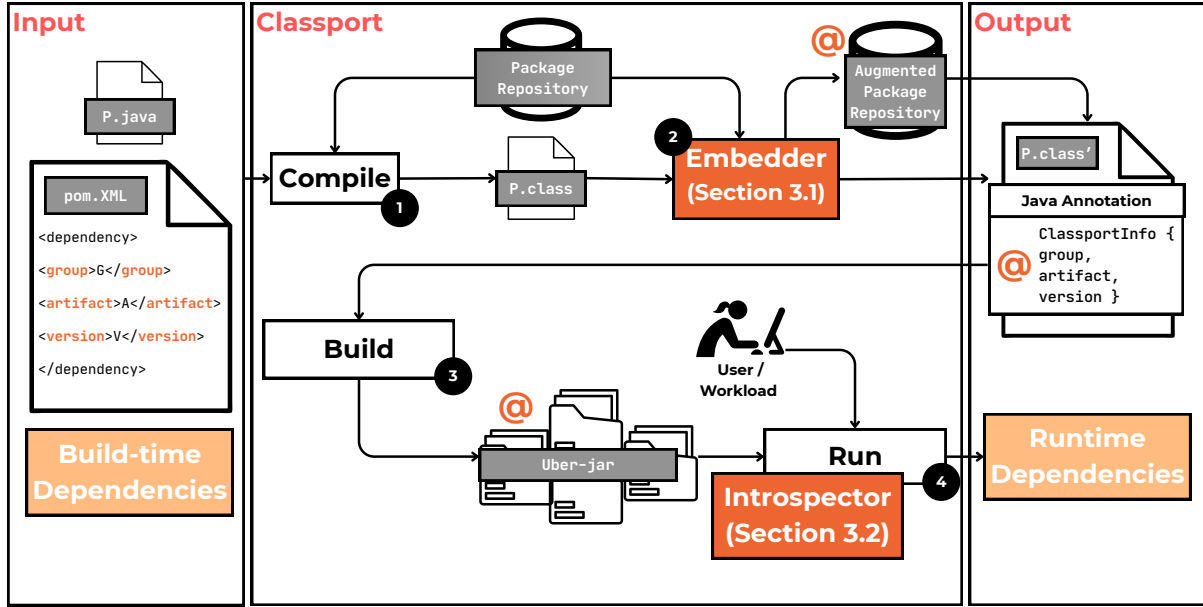


Figure 1: Overview of Classport, a novel system that enables runtime dependency introspection in Java.

and is uniquely identified by Group ID, Artifact ID, and Version [28], this triple being called a GAV coordinate. The Group ID represents the organization or project namespace, e.g., *org.apache.commons*. The Artifact ID specifies the module or library, e.g., *commons-lang3*. The Version distinguishes same library across different releases, e.g., *3.12.0*.

At build time, the dependencies of the `pom.xml` are resolved, in order to list the GAV coordinates of both direct and transitive dependencies. This allows Maven to download the exact version to use. In cases of conflicts between multiple versions of the same transitive dependency, Maven’s dependency resolution mechanism selects the version closest to the root of the dependency tree [27]. The version visible in the `pom.xml` is thus not necessarily the same one that will be used at runtime. In Maven, dependency metadata such as the GAV of dependencies is only available at build time, this information is not preserved at runtime at all [31]. The platform makes it impossible to identify the set of dependencies that are actually used at runtime, or to make any runtime decision depending on the dependencies. **In other words, Java supports classes, methods, and fields introspection, but not dependency introspection.**

In this paper, we address this problem by proposing a blueprint architecture and a prototype for supporting dependency introspection in Java.

3 Classport

We propose Classport, a novel technique for runtime dependency introspection in Java. Recall that in Java, dependency metadata is available at build time, but it is absent during execution.

We define **runtime dependencies** as dependencies that: 1) are fully specified within the binary to be executed 2) which are available as variables from within the program.

We also provide an open-source implementation of Classport. The main goal of Classport is to embed Software Supply Chain information, i.e., the dependencies, in the final executable artifact of a Java application, and extract this information at runtime, during application execution. Classport achieves this goal through its two key components: the Embedder and the Introspector. The Embedder stores the information about dependencies in Java artifacts, while the Introspector extracts this embedded information with runtime introspection.

Figure 1 presents an overview of Classport and its integration within the Maven lifecycle. Classport processes a target Java Maven project, which contains the source code and the information about build-time dependencies, which are listed in the `pom.xml` file. The Embedder performs the *embed* action (step 2), following the Maven compilation phase (step 1). This produces the Java binary artifacts enriched with the dependency information, which are represented in the output box. After the Maven build phase (step 3), the `uber-jar`, i.e., the executable archive comprising the project’s dependencies, is run (step 4). During this phase, the Introspector *introspects* the execution of the Java program and gives as output the runtime dependencies.

The following two subsections detail the technical aspects of the two components of Classport. Next, we present an illustration of Classport with an end-to-end example in Section 3.3. Finally, we give details about the implementation of Classport, including tools and libraries used, in Section 3.4.

3.1 Embedder

The **Embedder** is the first key component of Classport, designed to bridge the gap between build-time and runtime visibility of dependency information. In Java Maven projects, dependency metadata is explicitly declared in the `pom.xml` and fully resolved during the build phase, but this information is not preserved in the final application artifacts. The Embedder addresses this limitation by capturing build-time dependency metadata and embedding it directly into the compiled artifacts, making it accessible at runtime.

The Embedder takes as input the compiled artifacts of a target Maven project, i.e., the project's class files and the JARs of the project's dependencies. The compilation phase, step 1 in Figure 1, ensures that all dependencies are resolved and available locally. For each dependency JAR, the Embedder locates the local version that was downloaded and processes each file within the JAR. Depending on the resource it encounters, this processing involves different actions. For class files within a dependency JAR, i.e., compiled Java source files with `.class` extension, the Embedder embeds dependency metadata, i.e., its group, artifact, and version, using a bytecode transformation. The same process is applied to the application classes, such as `P.class` in Figure 1, annotated with the project module information. In Figure 1, the embedded information is represented by the `@` symbol.

When the Embedder encounters manifest files with a dependency JAR, such as `MANIFEST.MF` files, it removes signature-related entries, e.g., digest attributes, and copies the remaining content. This is necessary because modifying the contents of a signed JAR, such as adding metadata to class files, invalidates its signature. The Embedder ignores signature files and copies the rest of the files without modifications. After processing each dependency JAR, it repackages it into a new JAR. It saves it in a directory, which we refer to as the augmented package repository, as shown in Figure 1.

Finally, as shown in step 3 of Figure 1, the augmented package repository, i.e., the one containing the annotated dependencies, is used to build the final uber-jar with embedded software supply chain information.

The rest of this subsection describes how the Embedder is implemented.

3.1.1 Embedder as a Maven plugin. We use a Maven plugin to implement the Embedder.³ A Maven plugin is a reusable and highly configurable component that adds specific tasks or goals to Maven's build process. In particular, it is possible to decide in which build phase to execute it by configuring a Maven plugin goal.

Within Classport, we create the custom *embed* goal. We configure it to fulfill the following needs of the plugin: (1) to have class files available locally. This is fundamental because they have to be embedded. (2) To have both the project's and dependencies' class files. Dependencies must be resolved to also have the transitive ones. (3) To be run in the `GENERATE_RESOURCES` phase.⁴ This phase ensures that class files and dependencies are resolved early in the Maven life cycle to make the embedded resources available to the subsequent phases, such as compile, test, and package.

The first point is achieved by compiling the target project before executing the plugin. It guarantees that the class files are available

Listing 1: The classport Java Annotation for sorting dependency information within the code. The annotation is available at runtime through introspection.

```
1 @Retention(RetentionPolicy.RUNTIME)
2 public @interface ClassportInfo {
3     String group(); // G
4     String artefact(); // A
5     String version(); // V
6 }
```

for processing. The transitive dependency resolution and execution in the `GENERATE_RESOURCES` phase are achieved by adding it to the goal declaration.

3.1.2 Information embedded as Java Annotations. The primary objective of the tool is to introspect the dependency information during execution, so it is crucial to embed data in a manner that allows for retrieval at runtime. We use Java Annotations to add such information into the binary class files. We use bytecode transformation for that task. In each class, we create a custom annotation *ClassportInfo*, from the annotation type shown in Listing 1.

The annotation contains the GAV coordinate, i.e., group, artifact, and version of the dependency to which the class belongs. The Retention annotation [16], `@Retention(RetentionPolicy.RUNTIME)`, is used to make the *ClassportInfo* annotation visible at runtime. After the injection of Software Supply Chain information by the Embedder, the target application is packaged as a uber-jar.

3.2 Introspector

Once dependency metadata has been embedded, it has to be retrieved during execution. The **Introspector** component of Classport enables this by dynamically extracting the embedded dependency metadata at runtime. As illustrated in phase 4 of Figure 1, the Introspector introspects the running application, reading embedded annotations from running classes to recover dependency metadata. This process allows Classport to determine which dependencies are actually used during execution, providing a precise view of the runtime dependency set.

3.2.1 Introspector as Java agent using the Instrumentation API. The Introspector is a Java agent attached to the JVM. The goal of the Introspector is to add code that extracts the dependency annotations. To achieve this, we instrument the methods of the application under study using code transformation. Each time an instrumented method is executed, the extraction logic is triggered, storing the current dependency GAV in a set. To maintain a low overhead, not every method is instrumented. Since our objective is to have a list of used dependencies, once a dependency GAV is identified and stored in the set, the subsequent methods belonging to that dependency are no longer instrumented. This allows us to drastically reduce the overhead, without modifying the output of Classport. Finally, a shutdown hook writes the resulting set of dependency GAVs into a CSV file.

³<https://maven.apache.org/guides/introduction/introduction-to-plugins.html>

⁴<https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html>

Listing 2: Example of resulting Java bytecode after embedding process.

```

1 Classfile ../ExtractText.class
2   ...
3   Constant pool:
4   ...
5   #89 = Utf8           SourceFile
6   #90 = Utf8           ExtractText.java
7 +  #91 = Utf8           Lio/github/.../ClassportInfo;
8 +  #92 = Utf8           group
9 +  #93 = Utf8           org.apache.pdfbox
10  ...
11  Attributes:
12 +  RuntimeVisibleAnnotations:
13 +    @ClassportInfo(
14 +      group = "org.apache.pdfbox",
15 +      version = "3.0.4",
16 +      artefact = "pdfbox-tools",
17 +    )

```

3.3 End-to-end Example

Listing 2 shows the changes Classport makes on the original bytecode of the class ExtractText in the PDFBox project⁵. In particular, it shows the resulting bytecode modifications after running the Embedder. Classport adds an attribute in the attribute section (lines 12 to 17), which is the representation of the annotation. The keyword RuntimeVisibleAnnotations (line 12) indicates the retention policy that has been set, and that the annotation is visible at runtime. The remaining part of the section represents the key-value pairs of the annotation, i.e., the Group, Version, and Artifact. Finally, the constant pool section (lines 4-9) is consequently updated with the added values.

Listing 3 shows the CSV output content of the Introspector. It reports the list of dependencies used during the execution of the application. Specifically, the Group, Artifact, and Version are reported for each dependency.

Listing 3: Example of Introspector’s output for PDFBox workload execution. Each row is a triple of Group, Artifact, and Version.

```

1 org.apache.pdfbox, pdfbox, 3.0.4
2 org.apache.pdfbox, fontbox, 3.0.4
3 org.apache.pdfbox, pdfbox-io, 3.0.4
4 commons-logging, commons-logging, 1.3.4
5 info.picocli, picocli, 4.7.6
6 org.apache.pdfbox, pdfbox-tools, 3.0.4
7 org.apache.pdfbox, pdfbox-debugger, 3.0.4

```

3.4 Implementation

Classport is implemented in Java. It is made of two modules: the Embedder uses the ASM⁶ library to manipulate the bytecode in the class files, while the Introspector is a Java agent that uses the Instrumentation API [15]. The implementation is open-source and available on GitHub.

⁵<https://github.com/apache/pdfbox>

⁶<https://asm.ow2.io/>

Table 1: Study subjects considered in our experiments.

Project	Version	Deps.	Workload
PDFBox-app	3.0.4	12	Extract text from a PDF file
Certificate-ripper	2.4.1	5	Print the certificate of a website
mcs	0.7.3	4	Lookup dependency coordinates in Maven Central
batik	1.17	6	Convert an SVG to PNG
checkstyle	10.23.0	34	Lint a Java file
zxing	3.5.3	4	Decode 4 QR codes

4 Experimental Methodology

This section outlines the methodology for our study evaluating the implementation of Classport. First, it presents the research questions that guide our investigation and information about the dataset. Then, it provides a detailed explanation of how we address these research questions. Lastly, it shows the overall experimental setup.

4.1 Research Questions

We evaluate Classport to demonstrate its effectiveness in introspecting runtime dependencies in real-world applications. First, we assess its capability to embed dependency information into Java binary artifacts. Then, we test its behaviour during runtime execution and its ability to retrieve the actual set of running dependencies. Specifically, we investigate the following research questions:

RQ1: To what extent can Classport effectively embed dependencies into Java binary artifacts?

RQ2: To what extent does Classport support runtime inspection of dependencies?

4.2 Dataset

We aim at evaluating Classport with a dataset of real-world, maintained, open-source Java projects, which use Maven and span a range of application domains, dependency sizes, and usage scenarios. Our dataset includes six projects, with projects varying in complexity, and includes both lightweight tools and large frameworks such as checkstyle. The most active repository is zxing, which has over 30K stars and around 10K forks on GitHub. Table 1 lists each project along with its version, number of dependencies, and the workload we use to exercise the project. The number of dependencies is calculated directly by Maven. The workload is decided by considering a regular use case scenario for each application. We compile all projects for RQ1 and analyze the resulting artifact. For RQ2, we execute all projects to test the Introspector component.

4.3 Methodology of RQ1

The first research question aims to evaluate the Embedder. We assess the dependency completeness, class completeness, and the performance overhead of embedding dependency metadata into Java binary artifacts using Classport. This is done by statically analyzing the JAR file produced by Classport.

Dependency Completeness This property ensures that Classport embeds the uber-jar with dependency metadata of all dependencies. To assess this, we first run the Embedder on the dataset, generating the annotated uber-jars. We only consider dependencies that are needed for the application’s execution, excluding test-only

dependencies, as they end up in the uber-jar. Then, we analyze the uber-jars to read the annotations inside the binary artifacts. We consider the embedding process complete in terms of dependency metadata if the dependency metadata, i.e., each Group ID, Artifact ID, and Version, matches the ground truth retrieved by running the Maven command `mvn dependency:list` on the project under test.

Class Completeness We assess completeness by verifying that the annotation is embedded in each class file of every dependency in the project. Although the dependency can have files other than class files, such as build manifests and various configuration files [25], we only consider class files for annotation as they are the only ones that can be executed. To verify completeness, we first run the Embedder on the study subjects. Then, we analyse the uber-jar, looking inside each class file. If all Java class files are annotated with the Classport annotation, the embedding process is complete in terms of class metadata.

Build Time Overhead Build time overhead is measured by comparing the time taken to build the uber-jar with and without the Embedder. In detail, we run package phase of the Maven build process with and without the Embedder and measure the time taken for each run using the Unix time command. We compute the time difference between the two runs and report the percentage of overhead.

Disk Overhead Disk overhead is the additional space required to store the embedded dependency metadata. To measure the disk overhead, we compare the sizes of the uber-jar before and after embedding by using the Unix `stat` command. We run the same Maven command as in ‘Build Time Overhead’ to build the uber-jar with and without the Embedder and measure the disk usage for each run using the Unix `stat` command. We compute the difference in size of the Jar between the two runs and report the percentage of overhead.

4.4 Methodology of RQ2

The second research question evaluates the behavior of Classport at runtime, in particular, its capability to introspect runtime dependencies. The objective is to show that 1) Classport can correctly identify which dependencies are currently running and 2) the target application’s runtime behavior is not affected by Classport.

To evaluate the Introspector, it is necessary to first run the Embedder, to ensure that the embedding process is completed and also that all the dependency metadata is embedded into the uber-jar. This means that we assume a correct Embedder, as validated by RQ1, and we now assess the Introspector, which is the component dedicated to the runtime introspection.

Dynamic Correctness Dynamic correctness is satisfied when the runtime detected dependencies correspond to the ones actually used during the execution of the workload, i.e., the number of false negatives and false positives is equal to zero. First, we define a workload for each application, as shown in Table 1. The workload reflects a regular use case of the selected application. Then, we run the workload on the embedded project, which produces a list of runtime dependencies observed during the execution of the workload. We compare this list with the ground truth, i.e., the list of dependencies resulting from the `mvn dependency:list` command. If all the dependencies match, this means that Classport can correctly

Table 2: RQ1 – Static Completeness and Overhead of Embedding Process

Project	Completeness		Overhead	
	Dependencies	Classes	Time	Space
PDFBox-app	12/12	7,914/7,914	+18.86s	12%
Certificate-ripper	4/5	426/426	+2.68s	14%
mcs	4/4	557/557	+4.43s	10%
batik	6/6	3,679/3,679	+1.15s	29%
checkstyle	34/34	10,691/10,691	+6.18s	17%
zxing	4/4	717/717	+0.51s	11%

identify dependencies for this workload at runtime. In the case of missing dependencies from the list detected by Classport, this means that either the workload does not trigger the dependencies or that it is a false negative. To assess this, we modify the pom.xml of the project to only include dependencies that are detected by Classport. Specifically, we moved the dependencies not detected by Classport out of the runtime classpath. Finally, we repackage the application and rerun it with the same workload. If the application executes without errors and produces the same output as without Classport, this demonstrates the correctness of Classport in identifying all runtime dependencies that are necessary for the workload.

Overhead To measure the runtime impact of Classport, we use Java Microbenchmarking Harness (JMH) to benchmark the workloads with and without the Classport agent⁷. We prefer microbenchmarking over a simple time difference between executions because microbenchmarking helps recording the time of execution when the JIT compiler has fully optimized a Java application. Microbenchmarking ignores optimization time, which can lead to misleading performance results [7]. By accounting for Java Virtual Machine (JVM) optimizations such as JIT compilation and warm-up, JMH provides a more accurate assessment of the runtime overhead introduced by Classport.

Functional correctness For validating that Classport does not break correctness, we run the test suite of the target application and monitor for runtime errors or test failures caused by the agent.

4.5 General Setup

We run the experiments on a server with an Intel i9-10980XE CPU (36 cores), 125 GB RAM, using Java 17.0.15 and Maven 3.8.7. The dataset and experiment scripts are publicly available on GitHub.

5 Experimental Results

This section presents the results of the experiments as detailed in the section 4.

5.1 Results for RQ1

The results of RQ1 are summarized in Table 2. It presents the effectiveness of Classport in embedding dependency metadata into Java class files across the open-source projects tested. Each row of the table represents the results of a project of our dataset. The first

⁷<https://github.com/openjdk/jmh>

column reports the completeness of the embedding process considering two aspects. The first one is the number of dependencies embedded compared to the total number of dependencies in the project, and the second one is the number of class files containing the Classport annotation, compared to the total number of class files in the uber-jar. Then, the table reports the overhead introduced during the build phase by the Embedder. In detail, it reports the absolute time overhead, i.e., the time added by the embedding process to the build, and the space overhead, i.e., the percentage increase in the size of the uber-jar after embedding.

Consider the first row of the table, which corresponds to the PDFBox project. Classport successfully embeds all 12 dependencies of PDFBox-app, which is the total number of dependencies required by the application. The output uber-jar contains 7,914 class files, and all of them are annotated with the dependency metadata. The embedding process introduces an absolute time overhead of 18.86 seconds and a space overhead of 12% in the final uber-jar.

Classport successfully embeds all dependencies and class files in the projects we tested. Thus, 5 out of 6 projects are completely embedded, while Certificate-ripper has one dependency that is not embedded. This is because the dependency is only required for compilation and not for runtime, which is a characteristic known at build time by Maven, and hence it is not included in the uber-jar. Moreover, classes of uber-jars of all 6 projects, including the partially embedded project, are annotated with the dependency metadata. This confirms that the embedding process is complete in terms of adding dependency metadata.

The embedding process introduces a moderate performance cost during the build phase. The build time overhead ranges from 0.51 to 18.86 seconds of added time. The value is computed as the time required for the embedding plugin to be run. Considering the entire build process, this time can be negligible in the context of longer testing or packaging steps [6]. The space overhead also varies from 10% to 29% which is considered acceptable given the introduction of a novel feature - visibility into runtime dependencies. The space overhead depends upon the number of dependencies and identifier names in the core binary content. Certificate-ripper and mcs have about the same number of classes and the exact same number of dependencies embedded, yet the overhead is larger for Certificate-ripper due to longer identifier names.

Answer to RQ1: Our experiments show that Classport completely embeds dependency metadata into Java class files. The evidence is supported by 1) six diverse Maven projects 2) which reflect real-world usage and complexity. Build time and space overhead are an acceptable trade-off for getting a unique feature absent from the Java stack: visibility into runtime dependencies.

5.2 Results for RQ2

We now assess to what extent we are able to obtain the dependency information during execution. Table 3 summarizes the effectiveness of Classport in detecting dependencies at runtime across the evaluated applications. The table is divided into two parts: the first reports the results regarding the dynamic correctness, while the

Table 3: RQ2 – Dynamic Correctness and Overhead of Intro-spector

Project	Dynamic correctness				Runtime Impact	
	TP	FP	TN	FN	Overhead	Functional Correctness
PDFBox-app	7	0	5	0	0.74%	✓
Certificate-ripper	4	0	0	0	4.27%	✓
mcs	3	0	1	0	1.22%	✓
batik	2	0	4	0	1.24 %	✓
checkstyle	5	0	29	0	0.81%	✓
zxing	3	0	0	0	4.02%	✓

second shows the runtime impact. Regarding the correctness, the introspection process is defined as correct if the set of runtime dependencies corresponds to the actually used during the execution of the workload, Section 4.4. Specifically, the first column reports the number of true positives (TP), which is the number of dependencies detected by Classport, that are also present in the ground truth, i.e., the dependencies resulting from the `mvn dependency:list` command. The second column reports the number of false positives (FP), which represents the dependencies that Classport finds but are not present in the ground truth. The third one is about true negatives (TN), i.e., the dependencies that are present in the ground truth but not detected by our tool because the workload does not trigger them. Finally, the fourth column reports the number of false negatives (FN), i.e., the number of dependencies missed by Classport. The part of the table related to the runtime impact is divided into two columns: one reports the percentage of overhead, and the other indicates whether the functional correctness of the target project is preserved after running Classport. For instance, in the case of mcs, Classport successfully detects three dependencies, which are true positives. The one that it does not detect is a true negative because the workload does not trigger it. Classport introduces an overhead of 1.22% and preserves the functional correctness.

Detected Runtime Dependencies The correctness of the identified runtime dependencies is confirmed for all the tested applications. For certificate-ripper and zxing, the workload covers the entire set of dependencies. For the other applications, not all static dependencies are runtime-detected. To investigate whether this is a false negative, we remove the dependencies from the runtime classpath that are not detected by the workload, as explained in subsection 4.4. For PDFBox and checkstyle, the workloads succeed, confirming that Classport correctly identifies the dependencies covered by the workload.

For mcs and batik, the workload fails because one class of removed dependencies is loaded. Manual investigation of the code reveals that these classes are loaded but not executed. This is due to the internal logic of the JVM that requires loading classes even if they are not used.⁸ This behavior is due to different reasons; in our case, this is related to the following two cases. The JVM loads classes when they are referenced in the code, such as when an object is assigned to a specific type, requiring the JVM to verify that type for safety, even if no actual methods are invoked. Similarly, classes for declared exceptions are loaded, regardless of whether

⁸<https://docs.oracle.com/javase/specs/jvms/se17/html/jvms-5.htm>

those exceptions ever occur during execution. It would be possible to modify Classport to also intercept the class loading event, but we aim at only detecting executed code. Overall, Classport correctly detects the dependencies used at runtime.

Overhead. The overhead during the execution of the workload is negligible for all the study subjects. It varies from 0.74% to 4.27% which shows that reporting the runtime dependencies does not affect the performance of the application. The overhead is due to the fact that Classport needs to instrument each method invocation and retrieve the dependency metadata from the class files.

Functional correctness We check that Classport does not break any functionality by running the test suite. For all the applications in the dataset, no functional divergence is observed. This means that the bytecode instrumentation done by Classport has no negative effects on the target application.

Answer to RQ2: Classport correctly introspects runtime dependencies. It does not affect application behavior. The execution overhead is negligible for all the study subjects. To our knowledge, Classport is the first approach to embed and retrieve dependencies at runtime in the Java ecosystem.

6 Use Cases of Classport

This section highlights two possible applications of Classport.

6.1 Runtime Permissions per Dependency

Limiting what resources (e.g., network, filesystem) software components can access at runtime is relevant for security. In the context of Software Supply Chain security, a promising strategy is to assign permissions per dependency, ensuring that each third-party library is granted only the privileges it strictly requires. This permission enforcement per dependency requires precise identification of dependencies at runtime [1, 4].

Classport facilitates this approach in Java by retrieving dependency metadata (Group ID, Artifact ID, and Version) at runtime, enabling permission managers to make security decisions based on the dependency's inherent characteristics.

6.2 Vulnerability Detection at Runtime

Vulnerability detection tools aim to identify known vulnerabilities in third-party libraries. Identifying dependencies that are actually used at runtime is fundamental to avoid false positives or false negatives in vulnerability assessment [30].

Classport addresses this gap by making it possible to observe exactly which dependencies are executed during a program's run. By embedding GAV metadata into class files and retrieving it dynamically, Classport enables runtime-aware vulnerability detection tools that focus only on actually used components.

7 Related Work

In this section, we position our contribution with respect to related work on embedding dependency metadata into executables and identifying runtime dependencies in Java.

7.1 Embedding Information into Executables and Binaries

Boucher et al. [3] propose a novel approach to identifying software dependencies, introducing ABOM, Automatic Bill Of Material. The main idea of ABOM is to query an executable for their dependency to facilitate vulnerability detection. ABOM automatically embeds hashes of dependencies into compiled binaries during the build process. These hashes are stored efficiently in a probabilistic data structure called Compressed Bloom Filters. It allows for rapid querying to detect the presence of specific dependency and hence to identify the vulnerabilities associated with it. However, since the data structure is probabilistic in nature, it can report presence of a dependency even if it is not actually present.

Seshadi et al. [23] present OmniBOR, which embeds a content-based identifier called gitoid into executables, including Java class files, to link them to an external Artifact Dependency Graph built during compilation. This approach requires build-time instrumentation and external storage. In contrast, our method embeds Maven GAV metadata directly into class files, allowing immediate runtime access via a Java agent without relying on external resolution. Our technique is dedicated to Java, and uniquely supports lightweight, runtime introspection of runtime dependencies.

Quatch et al. [21] propose a solution that combines static and dynamic analysis to identify and remove the unused code during the program loading process. Their framework generates function-level dependency graphs and embeds this information into an optional section of the ELF binary. This embedded dependency information is then used by a piece-wise loader at runtime to identify and remove unused code from the program's memory. Our approach is more general and is not limited to use cases like debloating.

7.2 Runtime Identification of Dependencies

In this section, we report works related to the runtime identification of dependencies.

Ponta et al. [19] present a novel code-centric and usage-based method for detecting, assessing, and mitigating vulnerabilities in open-source software dependencies. This work builds upon their previous approaches [17, 18], and is implemented in the open-source tool Eclipse Steady.⁹ They use a dynamic call graph of dependencies to check the reachability of the vulnerable code. They instrument the target Java application and search if, during the execution, it reaches the vulnerable code, i.e., if it uses vulnerable methods. The difference with our approach is that they focus on the vulnerable dependencies and not on an overall identification of the actually executed dependencies.

Soto-Valero et al. [26] propose a tool to detect bloated code in Java and to remove it by transforming the bytecode of the compiled project. They use four code coverage tools along with probes to identify methods that are not executed during runtime. If no methods in any class are executed within a dependency, the dependency is considered bloated. They only work at testing time. However, we embed dependency information directly in the class file in JARs, making this information available at production time as well.

Xu et al. [32] propose a framework to find the causes of bloat within Java programs. Their framework utilizes abstract dynamic

⁹<https://github.com/eclipse-steady/steady>

slicing, a scalable technique that analyzes dynamic dataflow over limited abstract domains to identify the roots of this inefficiency. This framework helps programmers pinpoint performance bottlenecks related to data handling.

Amusuo et al. [1] implement a tool that is able to create a policy file with a mapping of required permissions for every dependency. The policies are then enforced at runtime. To create a policy file, the tool maps the Java package names inside a dependency with the required permissions. The limitation with this approach is that package names can be renamed when reused in dependent software [9]. Our approach overcomes this key limitation by embedding the dependency information into every class file which is immutable.

8 Conclusion

In this paper, we proposed Classport, a novel approach to introspect runtime dependencies in Java. Classport embeds dependency information into Java binary artifacts and exposes this information at runtime, during execution. We demonstrated its effectiveness by introspecting six real-world, open-source Maven Java projects. For all of them, it is possible to fully embed dependency information and to correctly identify the executed dependencies at runtime, with negligible overhead.

As future work, we plan to harden Classport against tampering by malicious Java agents. Since dependency metadata is embedded as annotations, it can be modified at runtime by agents that intercept class loading. To address this, we propose to use cryptographic guarantees by signing the class files, including dependency annotations, at build time. Then, one could verify the signature at load time and run time. This would ensure the integrity of dependency metadata as far as runtime.

References

- [1] Paschal C. Amusuo, Kyle A. Robinson, Tanmay Singla, Huiyun Peng, Aravind Machiry, Santiago Torres-Arias, Laurent Simon, and James C. Davis. 2024. ZTD_{JAVA}: Mitigating Software Supply Chain Vulnerabilities via Zero-Trust Dependencies. arXiv:2310.14117 [cs.CR] <https://arxiv.org/abs/2310.14117>
- [2] Musard Balliu, Benoit Baudry, Sofia Bobadilla, Mathias Ekstedt, Martin Monperrus, Javier Ron, Aman Sharma, Gabriel Skoglund, César Soto-Valero, and Martin Wittlinger. 2023. Challenges of Producing Software Bill of Materials for Java. *IEEE Security & Privacy* 21, 6 (2023), 12–23. <https://doi.org/10.1109/MSEC.2023.3302956>
- [3] Nicholas Boucher and Ross Anderson. 2023. Automatic Bill of Materials. arXiv:2310.09742 [cs.CR] <https://arxiv.org/abs/2310.09742>
- [4] Carmine Cesarano, Martin Monperrus, and Roberto Natella. 2025. GoLeash: Mitigating Golang Software Supply Chain Attacks with Runtime Policy Enforcement. arXiv:2505.11016 [cs.CR] <https://arxiv.org/abs/2505.11016>
- [5] Serena Cofano, Giacomo Benedetti, and Matteo Dell’Amico. 2024. SBOM Generation Tools in the Python Ecosystem: an In-Detail Analysis. In *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 427–434.
- [6] Oscar Cornejo, Daniela Briola, Daniela Micucci, Davide Ginelli, Leonardo Mariani, Adrián Santos Parrilla, and Natalia Juristo. 2024. A family of experiments about how developers perceive delayed system response time. *Software Quality Journal* 32, 2 (2024), 567–605.
- [7] Diego Costa, Cor-Paul Bezemer, Philipp Leitner, and Artur Andrzejak. 2021. What’s Wrong with My Benchmark Results? Studying Bad Practices in JMH Benchmarks. *IEEE Transactions on Software Engineering* 47, 7 (2021), 1452–1467. <https://doi.org/10.1109/TSE.2019.2925345>
- [8] Cybersecurity and Infrastructure Security Agency. 2024. Software Bill of Materials (SBOM). <https://www.cisa.gov/sbom>.
- [9] Jens Dietrich, Shawn Rasheed, Alexander Jordan, and Tim White. 2024. On the Security Blind Spots of Software Composition Analysis. In *Proceedings of the 2024 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED ’24)*. Association for Computing Machinery, New York, NY, USA, 77–87. <https://doi.org/10.1145/3689944.3696165>
- [10] The Apache Software Foundation. [n.d.]. Maven. <https://maven.apache.org/>
- [11] Aaron Linskins. 2024. The scale of open source: Growth, challenges, and key insights. <https://www.sonatype.com/blog/the-scale-of-open-source-growth-challenges-and-key-insights>
- [12] National Telecommunications and Information Administration. 2021. Software Bill of Materials (SBOM). <https://www.ntia.gov/page/software-bill-materials>.
- [13] Chinenye Okafor, Taylor R. Schorlemmer, Santiago Torres-Arias, and James C. Davis. 2022. SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties. In *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (Los Angeles, CA, USA) (SCORED ’22)*. Association for Computing Machinery, New York, NY, USA, 15–24. <https://doi.org/10.1145/3560835.3564556>
- [14] OpenSSF and The Linux Foundation. 2022. The Open Source Software Security Mobilization Plan. <https://openssf.org/oss-security-mobilization-plan>.
- [15] Oracle. 2023. Instrumentation API. <https://docs.oracle.com/en/java/javase/21/docs/api/java.instrument/java/lang/instrument/Instrumentation.html>
- [16] Oracle. 2023. Retention. <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/annotation/Retention.html>
- [17] Henrik Plate, Serena Elisa Ponta, and Antonino Sabetta. 2015. Impact assessment for vulnerabilities in open-source software libraries. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 411–420. <https://doi.org/10.1109/ICSME.2015.7332492>
- [18] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2018. Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 449–460. <https://doi.org/10.1109/ICSME.2018.00054>
- [19] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2020. Detection, assessment and mitigation of vulnerabilities in open source dependencies. *Empirical Software Engineering* 25, 5 (2020), 3175–3215.
- [20] Piotr Przytycki and Thomas Durieux. 2025. Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 91–102.
- [21] Anh Quach, Aravind Prakash, and Lok Yan. 2018. Debloating software through {Piece-Wise} compilation and loading. In *27th USENIX security symposium (USENIX Security 18)*. 869–886.
- [22] Md Fazle Rabbi, Arifa Islam Champa, Costain Nachuma, and Minhaz Fahim Zibran. 2024. Sbom generation tools under microscope: A focus on the npm ecosystem. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. 1233–1241.
- [23] Bharathi Seshadri, Yongkui Han, Chris Olson, David Pollak, and Vojislav Tomašević. 2024. OmniBOR: A System for Automatic, Verifiable Artifact Resolution across Software Supply Chains. arXiv:2402.08980 [cs.SE] <https://arxiv.org/abs/2402.08980>
- [24] Ax Sharma. 2021. What You Need to Know About the Codecov Incident: A Supply Chain Attack Gone Undetected for 2 Months. <https://www.sonatype.com/blog/what-you-need-to-know-about-the-codecov-incident-a-supply-chain-attack-gone-undetected-for-2-months>
- [25] Aman Sharma, Benoit Baudry, and Martin Monperrus. 2025. Canonicalization for Unreproducible Builds in Java. <https://doi.org/10.48550/arXiv.2504.21679> arXiv:2504.21679 [cs]
- [26] César Soto-Valero, Thomas Durieux, Nicolas Harrand, and Benoit Baudry. 2023. Coverage-Based Debloating for Java Bytecode. *ACM Trans. Softw. Eng. Methodol.* 32, 2, Article 38 (April 2023), 34 pages. <https://doi.org/10.1145/3546948>
- [27] The Apache Software Foundation. 2024. Introduction to the Dependency Mechanism. <https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html>.
- [28] The Apache Software Foundation. 2024. Introduction to the POM. <https://maven.apache.org/guides/introduction/introduction-to-the-pom.html>.
- [29] Ilkka Turunen. 2021. Log4Shell by the numbers: Why did CVE-2021-44228 set the internet on fire? <https://www.sonatype.com/blog/why-did-log4shell-set-the-internet-on-fire>
- [30] Laurie Williams, Giacomo Benedetti, Sivana Hamer, Ranindya Paramitha, Imranur Rahman, Mahzabin Tamanna, Greg Tystahl, Nusrat Zahan, Patrick Morrison, Yasemin Acar, Michel Cukier, Christian Kästner, Alexandros Kapravelos, Dominik Wermke, and William Enck. 2025. Research Directions in Software Supply Chain Security. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 146 (May 2025), 38 pages. <https://doi.org/10.1145/3714464>
- [31] Yue Xiao, Dhillung Kirat, Douglas Lee Schales, Jiyong Jang, Luyi Xing, and Xiaojing Liao. 2025. JBomAudit: Assessing the Landscape, Compliance, and Security Implications of Java SBOMs. In *ISOC Network and Distributed System Security Symposium*.
- [32] Guoqing Xu, Nick Mitchell, Matthew Arnold, Atanas Rountev, Edith Schonberg, and Gary Sevičky. 2014. Scalable Runtime Bloat Detection Using Abstract Dynamic Slicing. *ACM Trans. Softw. Eng. Methodol.* 23, 3, Article 23 (June 2014), 50 pages. <https://doi.org/10.1145/2560047>
- [33] Sheng Yu, Wei Song, Xunchao Hu, and Heng Yin. 2024. On the correctness of metadata-based SBOM generation: A differential analysis approach. In *2024 54th*

Classport: Designing Runtime Dependency Introspection for Java

Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). IEEE, 29–36.